



---

# Advanced ML

## Practical work 3

---

### Authors

| Name                   | UPC e-mail                                             |
|------------------------|--------------------------------------------------------|
| Katrine Bukh Villesen  | <code>katrine.bukh.villesen@estudiantat.upc.edu</code> |
| Daniel Reverter Condal | <code>daniel.reverter@estudiantat.upc.edu</code>       |
| Marc Parcerisa Conesa  | <code>marc.parcerisa@estudiantat.upc.edu</code>        |

January 16, 2026

## Abstract

This third practical work concludes our exploration of the ATLAS Higgs Boson Machine Learning Challenge dataset by investigating the performance and scalability of the recently proposed Kolmogorov-Arnold Networks (KANs). Building upon our previous analyses of Generalized Linear Models (GLMs) and Kernel-based classification methods, this study performs a systematic comparison between shallow KANs and traditional Multi-Layer Perceptrons (MLPs). We evaluate the neuron efficiency of both architectures by training a wide range of models with logarithmically varying widths and comparing their predictive power using Approximate Median Significance (AMS), AUC, and accuracy. While MLPs rely on fixed activation functions and learnable weights, KANs utilize learnable activation functions on edges, theoretically offering higher expressivity with fewer parameters. Our empirical results, however, discover significant trade-offs in this approach. We observe that while KANs can achieve competitive performance, they exhibit distinct scaling behaviors regarding computational cost and training stability. Specifically, we identify a divergence point where the time-to-convergence for KANs increases disproportionately compared to MLPs, suggesting that the theoretical efficiency of KANs may come at a high computational price for larger widths in some classification tasks.

# 1 Introduction

Back in 2024, a team from the Massachusetts Institute of Technology proposed a new technology they named Kolmogorov-Arnold Networks (KAN) for building neural networks that rather than learning a set of weights for each edge and having fixed activation functions on the nodes, have learnable activation functions on the edges, and nodes that simply perform a sum operation [4]. In their simpler form, those activation functions are 1D real functions parametrized as a spline, however, since then, more complex activation functions have been proposed, such as Chebyshev Polynomials [6] or Radial Basis Functions [3].

Even though replacing all weights of a Multi-Layer Perceptron with a spline could make it prohibitively expensive to optimize, fortunately, KANs allow much shallower and narrower computation graphs than MLPs [4]. This study aims at bridging the gap between the expressivity of MLP neurons and that of KAN nodes by systematically comparing the achieved performance of shallow MLPs and KANs with varying amount of neurons. To do so, we trained dozens of MLP networks and KAN networks on the Higgs Boson Challenge Dataset from 2014, and recorded the results, which are presented in [section 5](#).

## 1.1 Kolmogorov-Arnold Representation Theorem

Whilst Multi-Layer Perceptrons are inspired by the universal approximation theorem, KANs focus on the Kolmogorov-Arnold representation theorem. Vladimir Arnold and Andrey Kolmogorov demonstrated that if  $f$  is a multivariate continuous function on a bounded domain, then  $f$  can be written as a finite composition of continuous functions of a single variable and the addition operator. Specifically, for  $f : [0, 1]^n \rightarrow \mathbb{R}$ ,

$$f(\mathbf{x}) = f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \psi_q \left( \sum_{p=1}^n \phi_{q,p}(x_p) \right), \quad (1)$$

where  $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$  and  $\psi_q : \mathbb{R} \rightarrow \mathbb{R}$ . Note, however, that the existence of such functions does not imply that they are easy to find, as they might be non-smooth, or even fractal, and not learnable in practice [5, 1]. Nonetheless, most functions in science and daily life are often smooth, which potentially makes them candidates for Kolmogorov-Arnold representations. The original paper from 2024, [4], goes one step further, and proposes deeper computational graphs for KANs, and generalizes the representation theorem to arbitrary widths and depths, rather than the  $2n + 1$  hidden terms in [Equation 1](#).

## 1.2 Kolmogorov-Arnold Networks

In the general case, a KAN can have arbitrary depth  $L$ , with each layer's width specified by  $[n_1, n_2, \dots, n_L]$ . Each layer's node values are computed by

$$x_{l+1,j} = \sum_{i=1}^{n_l} \phi_{l,j,i}(x_{l,i}), \quad \text{with } j = 1, \dots, n_{l+1}, \quad (2)$$

where  $\phi_{l,j,i}$  is the activation function placed on the edge that goes from neuron  $i$  in layer  $l$  to node  $j$  in layer  $l + 1$ . We specify, then, that  $x_{0,*}$  is the input vector  $\mathbf{x}$ . See [Figure 1](#) for a graphical example. Notice that the Kolmogorov-Arnold representation theorem at [Equation 1](#) is a specific case of a 3-layer-deep (two sets of non-linear edges) KAN with a width vector  $[n, 2n + 1, 1]$ , where there is a single hidden layer with width  $2n + 1$ .

The recursivity at [Equation 2](#) is equivalent to a composition of several function matrices of the form

$$\mathbf{x}_{l+1} = \begin{pmatrix} \phi_{l,1,1} & \phi_{l,2,1} & \cdots & \phi_{l,n_l,1} \\ \phi_{l,1,2} & \phi_{l,2,2} & \cdots & \phi_{l,n_l,2} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,1,n_{l+1}} & \phi_{l,2,n_{l+1}} & \cdots & \phi_{l,n_l,n_{l+1}} \end{pmatrix} \mathbf{x}_l,$$

which we denote as  $\Phi_l$ . Thus, a KAN is computed as

$$\text{KAN}(\mathbf{x}) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \cdots \circ \Phi_1 \circ \Phi_0) \mathbf{x}. \quad (3)$$

Because absolutely all the steps only involve differentiable functions, a KAN can be fitted using backpropagation.

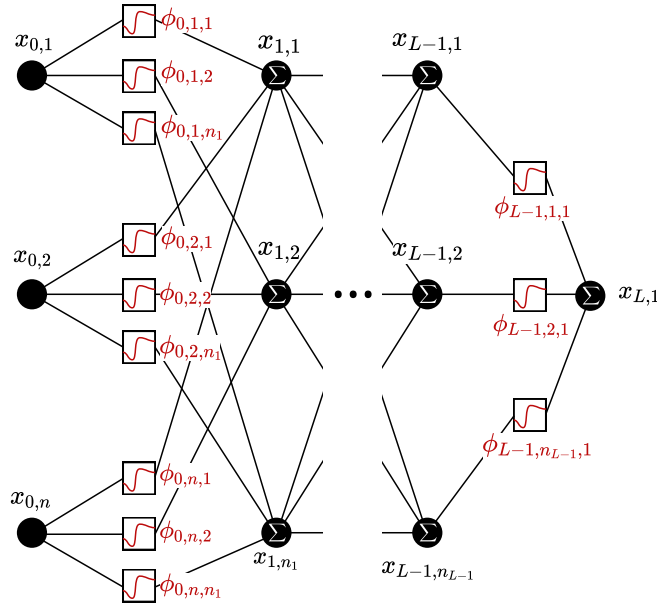


Figure 1: Graphical representation of a Kolmogorov-Arnold Network with  $n$  inputs and one output, along with the notation to represent the activation functions and node values.

## 2 Related Work

In the original publication, KANs are compared to MLPs in terms of their accuracy (loss) and their complexity (number of parameters), and shown to display more favorable Pareto Frontiers than MLPs [4].

In [7], authors control the number of parameters and FLOPs of KANs and MLPs across tasks such as computer vision, audio processing, natural language processing and symbolic formula representation, and observe that, except for symbolic formula representation tasks, MLPs generally outperform KANs. They also discover that, when applying B-spline to MLPs, their performance in symbolic formula representation significantly increases, suggesting that the sole advantage of KANs over MLPs for that task relies on their B-splines.

In contrast, in [8], the authors vary the number of parameters and the size of the training samples to compare the performance of MLPs and KANs across regular functions, continuous functions with local non-differentiable points, functions with jump discontinuities, functions with singularities, functions with coherent oscillations, and noisy functions. They find that KANs not always perform best, however they tend

to converge faster than MLPs across all functions. They also find that the Adam optimizer mostly exceeds L-BFGS in performance, which inspired us to use Adam as the optimizer for our experiments.

Finally, and quasi-comically, the authors of [2] disregard any comparison between the two, and implements a Mixture-of-Experts architecture which employs both an MLP and a KAN to dynamically adapt to the specific characteristics of the task at hand.

### 3 Data and pre-processing

The analysis employs the same Higgs boson classification dataset that was also used in the first two practical works. It consists of labeled instances for distinguishing signal events from background events in high-energy physics experiments.

Some pre-processing of the data is done similar to the previous projects:

- **Feature Selection and Cleaning:** Relevant features are retained, while missing or corrupted values are addressed through a combination of instance and feature removal.
- **Normalization:** Continuous features are standardized to zero mean and unit variance to prevent scale-related bias in model training.
- **Remove correlated features:** Identify and remove highly correlated features using a Pearson correlation threshold of 0.85.

Maintaining identical pre-processing ensures that differences in model performance reflect the modeling techniques rather than variations in data preparation.

### 4 Methodology

Both the Kolmogorov–Arnold Network (KAN) and Multilayer Perceptron (MLP) experiments follow an identical experimental pipeline to ensure a fair and controlled comparison. For both models, a single hidden layer architecture is used and the width of this layer is systematically (logarithmically) varied while all other hyperparameters are kept fixed. The models are trained using the same optimization strategy and loss function, and four performance metrics are evaluated and registered on a tracking service after each training epoch on a test sample.

To rule out any effect of initialization of the models or the specific test splits used, and because at no point do we need to perform hyperparameter selection, instead of using a single train/test split, all experiments are run five times, with five different non-overlapping folds of the dataset as the test sample.

Thus, all experiments can be uniquely identified (and thus re-created) by a combination of the training hyperparameters (e.g. model used, hidden dimension, learning rate, etc.) and two more parameters representing how many folds the dataset is split into, and which fold is picked as the test sample. And all experiments perform the same steps:

1. The cleaned Higgs Boson dataset (specified above) is loaded.
2. A custom logic computes the indices of the training and testing set based on the fold amount and the test index, as specified above. The dataset is split into train and test.
3. A model is initialized and weights are determined at random but with a pre-defined seed.
4. Training follows as usual, in batches of training samples, the model is used to predict, the Binary Cross Entropy loss is computed, the Adam optimizer updates the weights, and the loop repeats until the end of the training data (one epoch).
5. After each epoch, Accuracy, AUC, AMS and the loss are evaluated over the test split, and their results are logged into MLflow.

6. The entire training loop repeats for a set amount of epochs.

This design makes sure that any difference in observed performance can be attributed to architectural differences rather than data handling, optimization, initialization, or artifacts from the evaluation.

## 4.1 System

All experiments were executed on a system with the following specifications:

- **CPU:** AMD Ryzen 9 9950X (16 Cores @ 4.30 GHz)
- **RAM:** 64GB @ 6000 MT/s
- **GPU:** Nvidia RTX3060 (14 GB)

## 4.2 Method 1: KAN

### 4.2.1 Implementation details

The KAN experiment employs a shallow Kolmogorov–Arnold Network with one hidden layer, where learnable functions are placed on the edges between neurons instead of using fixed activation functions at the nodes. A custom `ClassificationKAN` model was developed that adds a single fixed sigmoid activation function at the end of the computational graph. The explanation for this is simple: Because the task at hand consists of predicting the probability of each of the two classes for a given sample point (Bernoulli distribution), we either hope that the KAN naturally learns a sigmoid-like activation function on the last neuron, or we enforce it via adding a single extra layer. Because this doesn’t include any new trainable parameters, we argue that it cannot favor KANs over MLPs in any of the experiments.

## 4.3 Method 2: Multilayer Perceptron

### 4.3.1 Model formulation

The Multilayer Perceptron (MLP) used in this study is a standard feed-forward neural network composed of an input layer, a single hidden layer, and an output layer. Each hidden unit computes an affine transformation of its inputs followed by a fixed nonlinear activation function, while the output layer produces a scalar score used for binary classification. The model implements a function of the form

$$f(x) = (\sigma \circ (W_2 + b_2) \circ \text{ReLU} \circ (W_1 + b_1))(x) \quad (4)$$

where  $g$  denotes the hidden-layer activation function and  $\sigma$  is the output activation, a sigmoid, implicitly defined by the chosen loss, and  $(W_l + b_l)$  represents a fully-connected layer with a learnable bias  $b_l$ .

### 4.3.2 Theoretical properties and justification

MLPs with at least one hidden layer are known to be able to approximate any continuous function on a compact domain given sufficient width. This property makes shallow MLPs a natural baseline for studying capacity in neural networks. However, their expressive power is achieved through fixed activation functions combined with linear weight parameters that may require a large number of neurons to represent complex functions efficiently. In this study, the MLP serves as a well-understood reference architecture against which the expressive efficiency of Kolmogorov–Arnold Networks can be assessed, particularly in terms of how performance scales with hidden-layer width.

### 4.3.3 Implementation details

As in the KAN experiments, the hidden-layer width is varied across a predefined range logarithmically, and each configuration is trained independently from a random initialization with, again, a set seed. The same optimizer, learning rate, number of epochs, loss function, and, most importantly, number of folds as in the KAN experiments are used to maintain consistency. After training, model performance is measured on the same test set.

## 4.4 Optimization details

All models are trained using the Adam optimizer with a fixed learning rate and a fixed number of training epochs. No early stopping or hyperparameter tuning is performed, thus, as explained above, the choice to skip validation and instead use a k-fold testing. This choice ensures that differences in performance are attributed to model architecture rather than optimization dynamics.

## 4.5 Performance metrics

We use a set of different performance metrics to compare the models:

- **Approximate Median Significance (AMS):** A metric reflecting the statistical significance of detecting signal events, defined in the Kaggle competition. This metric captures subtle improvements in the signal-background separation that accuracy or AUC may miss.
- **Area Under the ROC Curve (AUC):** Measures ranking performance and is insensitive to class imbalance.
- **Accuracy:** The proportion of correctly classified samples.
- **Binary Cross Entropy:** The loss that was used and optimized for in all experiments.

## 4.6 Computational efficiency

In addition to predictive performance, we explicitly evaluate computational efficiency and model complexity. To assess each method’s scalability and practicality, we track:

- Epochs to convergence of the models, evaluating how many epochs did each training take until a convergence criteria was met (explained later).
- Time to convergence of the models, evaluating the wall-clock time that each model took to meet the same convergence criteria. This was proven to be very relevant when choosing between MLP and KAN.

# 5 Discussion of results

## 5.1 Overall performance

To sum up the performance of each model, [Table 1](#) presents the average of four different performance metrics evaluated over the five testing folds each model was trained with, for the most relevant models found in terms of the AUC reached, as well as two benchmark models for reference, among which is the winner of the Kaggle competition. For each of the models in this work we present the best performing one in this table and go deeper into the comparison later. For more details on the actual hyper-parameters used, and the results obtained, please refer to the code attached. In [Table 1](#) we see that all the models perform overall very well with high accuracy and ROC-AUC scores compared to the benchmark models.

| Model                    | AMS    | ROC-AUC | Accuracy | BCE    |
|--------------------------|--------|---------|----------|--------|
| <b>Our models</b>        |        |         |          |        |
| MLP (64 hidden neurons)  | 0.2751 | 0.8716  | 0.7992   | 0.4427 |
| KAN (128 hidden neurons) | 0.2776 | 0.8855  | 0.8103   | 0.4131 |
| <b>Benchmarks</b>        |        |         |          |        |
| Baldi et al. (DNN)       | -      | 0.88    | 0.81     | -      |
| Kaggle Winner (XGBoost)  | -      | 0.90    | -        | -      |

Table 1: Summary of each models performance compared to two benchmark models, over their respective test samples.

## 5.2 Detailed analysis

In this section we summarize the empirical findings obtained from the systematic comparison of KAN and MLP architectures. We compare models across four validation metrics (AMS, AUC, Accuracy, and Binary Cross Entropy) and study how these quantities and convergence behaviors depend on the model size and complexity.

### 5.2.1 Data Filtering and Convergence Criteria

We filtered the results to exclude models that failed to stabilize within the fixed training schedule. We utilized the full metric histories retrieved from MLflow to compute a convergence heuristic: the slope of a linear regression fitted over the last 5 epochs for each metric. The absolute slope thresholds for convergence were set as follows:

- Accuracy slope  $< 0.0015$
- AMS slope  $< 0.0005$
- AUC slope  $< 0.0015$
- Loss slope  $< 0.002$

Which were chosen somewhat arbitrarily after exploring the distribution of the "final slopes", which can be seen in 2, and making sure that roughly 90% of the experiments passed the threshold.



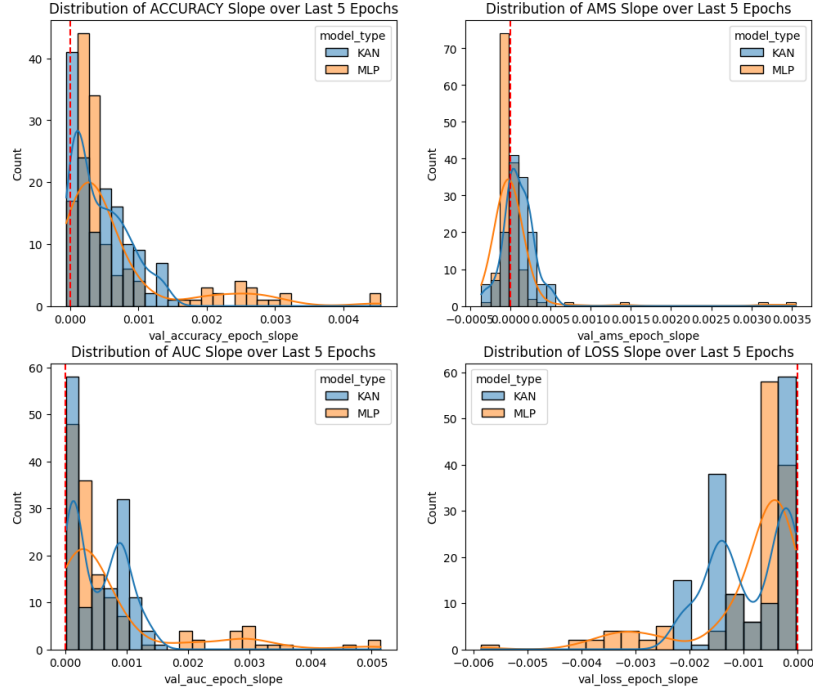


Figure 2: Distribution of the rate of change for validation metrics over the last 5 epochs

The vast majority of runs clustered near a slope of zero, indicating stability. Based on the defined thresholds, 38 runs out of 281 (approximately 13.5%) were labeled as non-converged and excluded. Analysis of these excluded runs revealed that they primarily corresponded to MLPs with extremely small hidden layers (1–2 neurons), which lacked the capacity to learn the problem, or specific KAN configurations that exhibited oscillating loss landscapes. See the code for a deeper exploration of convergence.

### 5.2.2 Predictive Performance vs. Parameter Efficiency

A central hypothesis of KANs is their parameter efficiency (the ability to achieve high expressivity with fewer parameters than MLPs). This could be falsely concluded if one looked at the misleading [Figure 3](#).

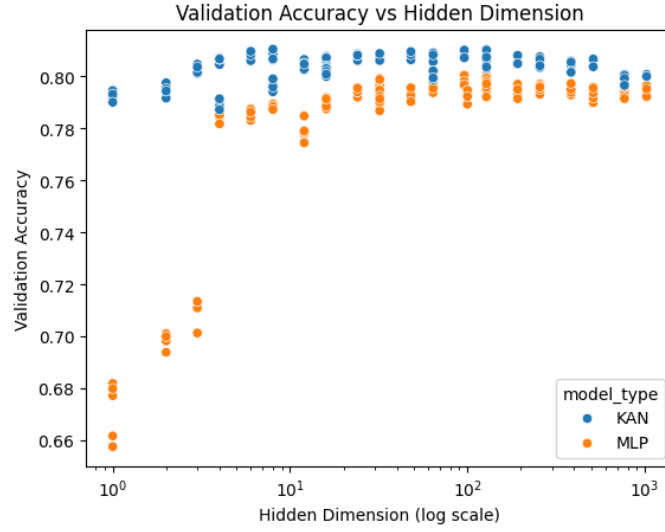


Figure 3: Results obtain for the dependency of the accuracy of the model with respect to its hidden dimension for both MLPs and KANs. Notice that KAN seems to always outperform MLP, however this is intentionally misleading, because KANs have many more learnable parameters per neuron.

However, most of the times, when talking about the expressivity of the networks, one would favor KANs over MLPs without knowing it, just because KANs have many more parameters than MLPs per neuron. For that reason, we decided to also analyze all metrics with respect to the number of parameters, which can be computed as follows.

For the specific MLP architecture used:

1. Input Layer: 17 features (the amount of features left in the dataset after cleaning)
2. Hidden Layer: As many neurons as defined by the experiment ( $H$ ). ReLU activation, which has no trainable parameters.
3. Output Layer: 1 neuron. Sigmoid activation, also without trainable parameters.

The total number of parameters can be calculated with

$$||\text{MLP}|| = (17 + 1) \cdot H + (H + 1) \cdot 1 = 19H + 1$$

For the specific KAN architecture used:

1. Input Layer: 17 features again.
2. Hidden Layer:  $H$  hidden neurons. Notice that the trainable parameters exist in the edges, rather than the neurons themselves, and that each edge has  $g$  grid intervals for the spline, of degree  $k$  and a scalable base function.
3. "Hidden" output layer: 1 neuron.
4. Output Activation: Sigmoid, without any trainable parameters. Explained above.

The total amount of parameters can be calculated with

$$||\text{KAN}|| = (17 \cdot H) \cdot (g + k + 1) + (H \cdot 1) \cdot (g + k + 1) = 18H \cdot (g + k + 2)$$

Thus, [Figure 4](#) presents how the metrics scale for both architectures in terms of their number of parameters.

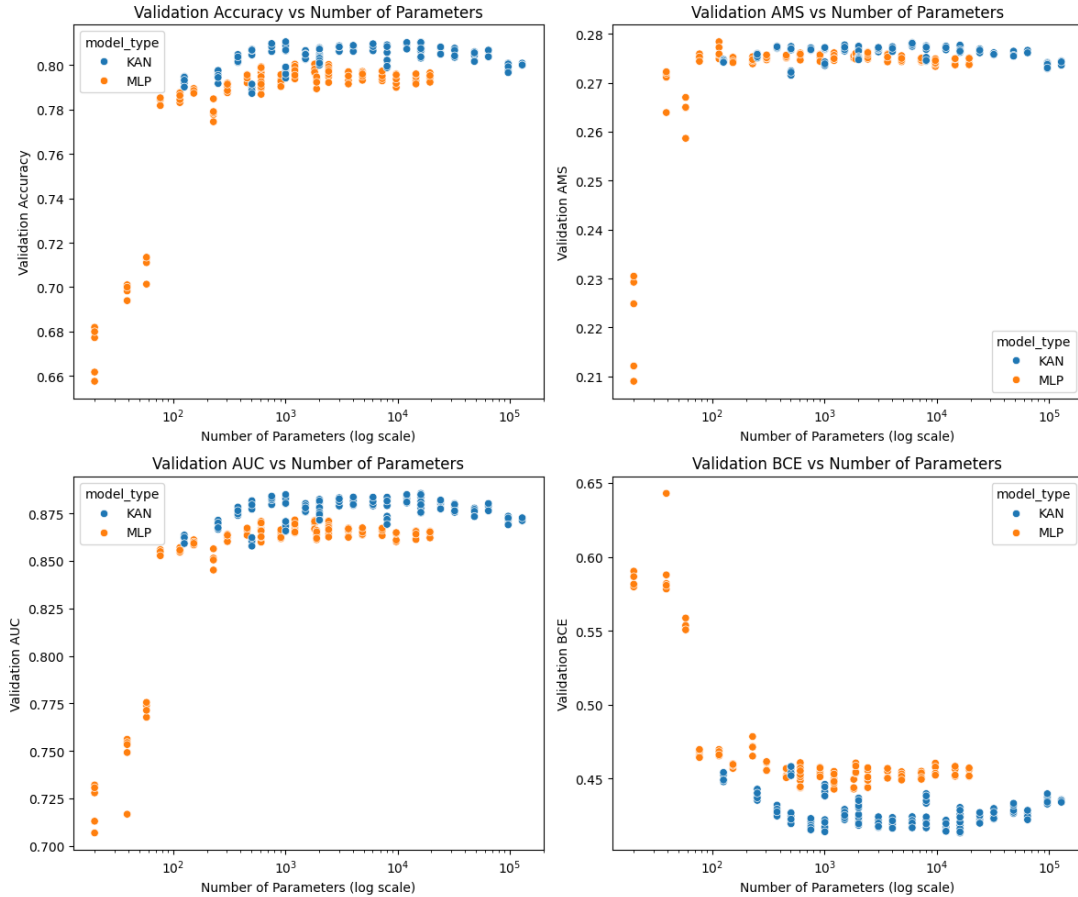


Figure 4: Dependency of all four validation Metrics vs. Total Number of Parameters. Notice that even though it does seem that KAN models perform slightly better than MLPs, their performance is much closer than when looking at the number of neurons.

The results clearly show that both models follow the expected general trend of diminishing returns: Performance improves rapidly as parameter count increases in the small-model regime, but eventually plateaus.

- **Very Small Scale:** In the very-low-parameter regime, we don't have any KAN result, as with a single hidden network, KANs already have many more parameters than MLPs do, however, one can see that MLP performance rapidly increases as they grow.
- **Small Scale:** For the low-parameter regime, one can see that even though KANs seem to achieve slightly better accuracy and AUC than MLPs, their performance is comparable.
- **Large Scale:** As models grow larger, the performance gap seems to widen a bit, although further exploration would be needed to conclude the significance of this result.

**Note about metric stability:** While AUC and Accuracy (left plots) show smooth monotonic improvements, AMS (top right) seems to plateau really fast for both models. This suggests AMS is less sensitive to fine-grained training dynamics.

### 5.3 Convergence Speed and Computational Cost

Perhaps the most significant finding relates to the computational cost of scaling these architectures. We analyzed the time to converge, measured both in epoch count as well as wall-clock time, against the model size, shown ?? and Figure 6 respectively.

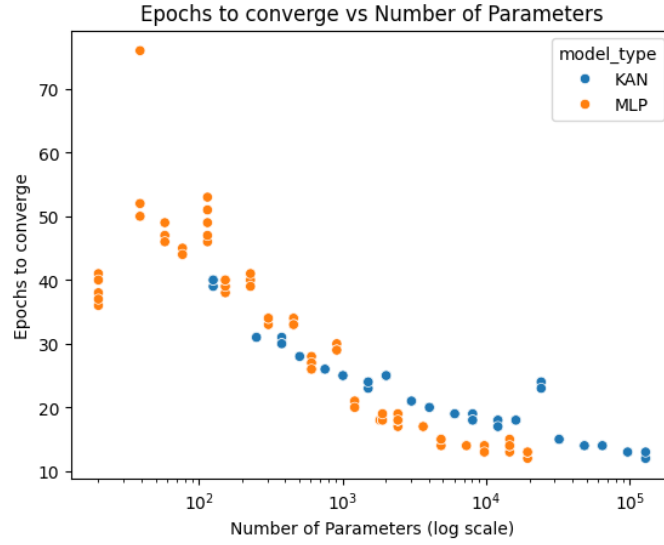


Figure 5: Epochs to Convergence vs. Number of Parameters (Log-Log Scale). Plot shows the expected behavior, as models grow, they take less and less epochs to converge. This has a lower bound that isn't explored in this report.

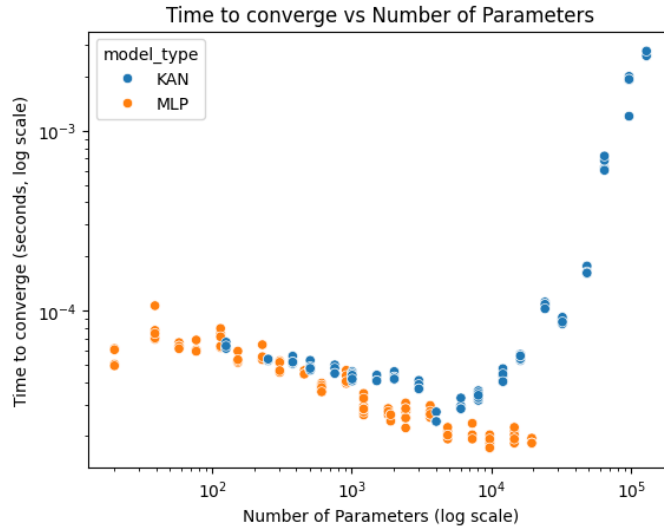


Figure 6: Wall-clock Time to Converge vs. Number of Parameters (Log-Log Scale). Unexpectedly, KANs start taking much longer times to converge as models grow, counteracting the analogous effect on the number of epochs.

When looking at the wall-clock time to convergence, both model families initially show a power-law relationship (linear on the log-log scale) between model size and training time. However, a critical divergence occurs for larger architectures, which in our system, specified above, occurred around 1300 parameters.

1. **MLPs:** The time to converge plateaus or scales gently, benefiting from highly optimized matrix multiplication operations on standard hardware.
2. **KANs:** The training time shoots up drastically beyond a certain parameter count. This implies that the computational complexity of optimizing KANs—likely due to the evaluation of B-spline activation functions on every edge—includes a significant non-trivial component.

While KANs may be parameter-efficient, they are not necessarily computationally efficient for wide, shallow architectures in some tasks compared to standard MLPs.

## 6 Conclusions

In this third practical work, the performance and scalability of Kolmogorov-Arnold Networks (KANs) was compared against traditional Multi-Layer Perceptrons (MLPs) on the task of classifying Higgs Boson events. By training a wide spectrum of model configurations with varying widths, we were able to assess the trade-offs between representational capacity, parameter efficiency, and computational cost.

The empirical results validate the theoretical promise of KANs regarding parameter efficiency in restricted regimes. In the low-parameter setting, KANs frequently achieved competitive or superior AMS and AUC scores compared to MLPs, supporting the hypothesis that learnable activation functions on edges provide higher expressivity per parameter than fixed node activations. However, as the model size increased, this performance gap narrowed, with both architectures eventually reaching a plateau of diminishing returns.

A critical divergence was observed regarding computational scalability. While MLPs exhibited a gentle, predictable scaling in convergence time due to highly optimized matrix operations, KANs displayed a drastic, non-linear increase in training costs for larger architectures. This suggests that the evaluation of B-spline functions introduces a significant computational overhead that outweighs the benefits of parameter reduction in large-scale, shallow networks.

Overall, while KANs prove to be a highly expressive and parameter-efficient architecture suitable for compact models, standard MLPs remain the more computationally robust choice for larger, high-throughput classification tasks. Future work could explore deeper KAN configurations or alternative activation approximations to mitigate the computational bottlenecks observed in wide, shallow architectures.

## References

- [1] Federico Girosi and Tomaso Poggio. Representation properties of networks: Kolmogorov’s theorem is irrelevant. *Neural Computation*, 1:465–469, 12 1989.
- [2] Yunhong He, Yifeng Xie, Zhengqing Yuan, and Lichao Sun. Mlp-kan: Unifying deep representation and function learning, 2024.
- [3] Ziyao Li. Kolmogorov-arnold networks are radial basis function networks, 2024.
- [4] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. Kan: Kolmogorov-arnold networks, 2025.
- [5] Tomaso Poggio, Andrzej Banburski, and Qianli Liao. Theoretical issues in deep networks: Approximation, optimization and generalization, 2019.
- [6] Sidharth SS, Keerthana AR, Gokul R, and Anas KP. Chebyshev polynomial-based kolmogorov-arnold networks: An efficient architecture for nonlinear function approximation, 2024.
- [7] Runpeng Yu, Weihao Yu, and Xinchao Wang. Kan or mlp: A fairer comparison, 2024.
- [8] Chen Zeng, Jiahui Wang, Haoran Shen, and Qiao Wang. Kan versus mlp on irregular or noisy functions, 2024.